CAFETERIA MEAL CATEGORY CLASSIFICATION

MIDTERM PROJECT

Presented by:

GROUP 13

Marquez, Arianne Beverly

Respicio, Lester Anthony

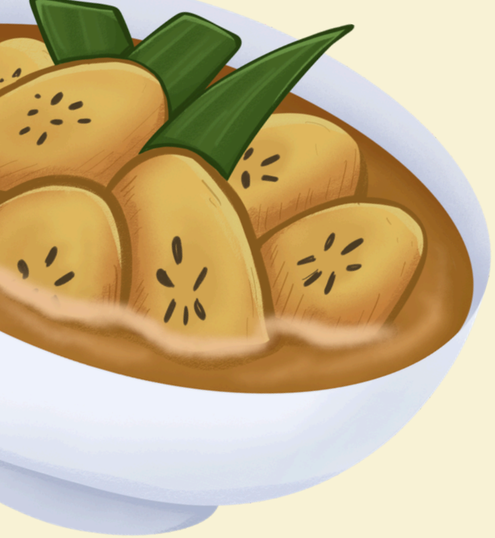
OUR TEAM

Arianne Beverly Marquez

Python Code & Streamlit Interface

Lester Anthony Respicio

CSV Data Formation & Knime Workflow



OVERVIEW

- SCENARIO
- CSV DATA
- KNIME (WORKFLOW & RESULTS)
- PYTHON
- ALGORITHM COMPARISON
- STREAMLIT INTERFACE
- TEST, EXPLAIN AND REFLECT

SCENARIO

A CAFETERIA WANTS TO CLASSIFY MEALS AS LIGHT, BALANCED, OR HIGH-CALORIE USING NUTRITIONAL INFORMATION.

csv

CSV

A	B	C	D	E	F	G	H
MEAL ID	MEAL NAME	CALORIES (kcal)	SUGAR (g)	FAT (g)	PROTEIN (g)	SODIUM (mg)	CATEGORY
MID-49	Corned Beef with Pandesal (2pcs)	420	5	18	16	850	HIGH-CALORIE
MID-50	Spam with Pandesal (2pcs)	480	5	25	15	1050	HIGH-CALORIE
MID-51	Siopao Asado (Large)	380	12	10	12	600	BALANCED
MID-52	Siopao Bola-bola (Large)	420	8	15	14	700	BALANCED
MID-53	Classic Ensaymada	350	15	18	6	300	BALANCED
MID-54	Cheese Roll	280	10	14	5	250	BALANCED
MID-55	Spanish Bread (1pc)	180	8	6	3	150	BALANCED
MID-56	Monay (1pc)	150	4	2	4	180	LIGHT
MID-57	Pan de Coco (1pc)	200	12	7	3	150	BALANCED
MID-58	Turon (Banana Spring Roll) (1pc)	220	15	8	2	50	BALANCED
MID-59	Banana Cue (1 stick)	250	25	5	2	10	HIGH-CALORIE
MID-60	Kamote Cue (1 Stick)	280	20	8	2	15	HIGH-CALORIE
MID-61	Maruya (Banana Fritter)	240	12	10	2	100	BALANCED
MID-62	Taho (Classic with Sago & Arnibal)	180	25	3	6	20	HIGH-CALORIE
MID-63	Puto Cheese (3 pcs)	150	10	2	4	250	BALANCED

CSV SPREADSHEET

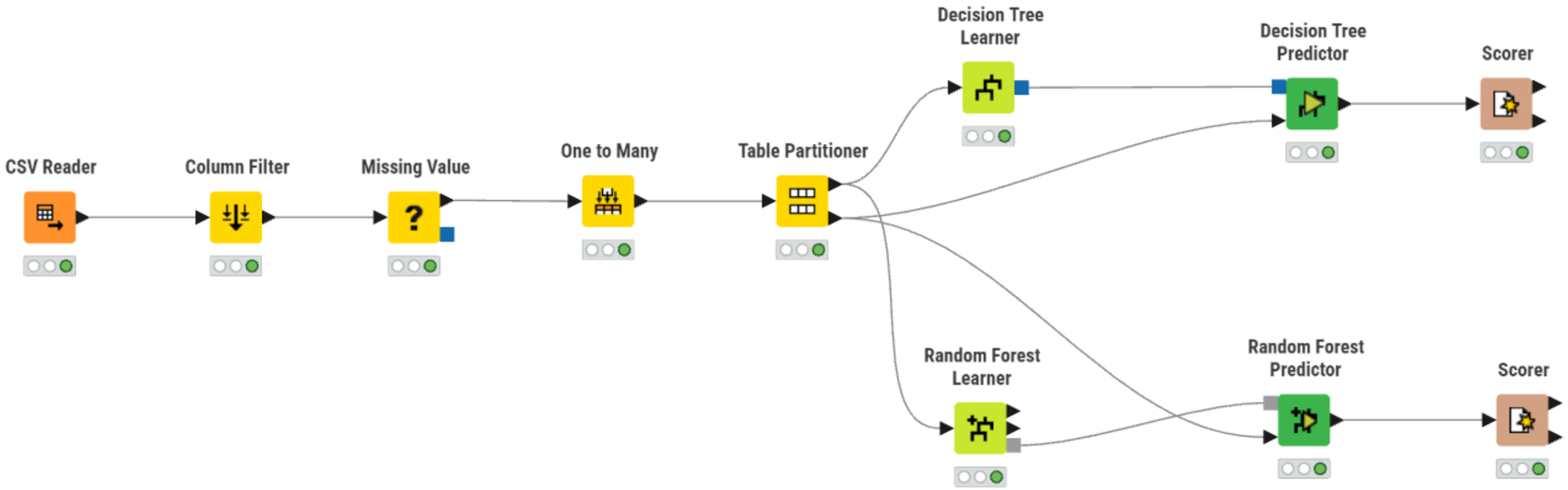
CSV FORMULA

```
=IFS(OR(C2>800, D2>15, E2>30, F2>40, G2>800), "HIGH-CALORIE",
AND(C2<400, D2<5, E2<10, F2<=35,G2<300), "LIGHT", TRUE,
"BALANCED")
```

- **HIGH-CALORIE** (OR CONDITION): TRIGGERED IF ANY SINGLE NUTRIENT SURPASSES UPPER THRESHOLDS (E.G., >800 KCAL, >30G FAT, >800MG SODIUM).
- **LIGHT** (AND CONDITION): ASSIGNED ONLY WHEN ALL NUTRITIONAL VALUES STAY STRICTLY BELOW LOW BASELINES (E.G., <400 KCAL, <10G FAT).
- **BALANCED** (DEFAULT): CAPTURES ALL INTERMEDIATE MEALS PROVIDING MODERATE PROPORTIONS.

KNIME

KNIME


CSV Reader
Column Filter
Missing Value
One to Many
Table Partitioner
Decision Tree Learner
Decision Tree Predictor
Scorer
Random Forest Learner
Random Forest Predictor
Scorer

WORKFLOW RESULTS

DECISION TREE SCORER

Rows: 4 | Columns: 11 Sorting Filter Search

#	RowID	TruePositiv... Number (Inte...	FalsePositi... Number (Inte...	TrueNegati... Number (Inte...	FalseNegat... Number (Inte...	Recall Number (Float)	Precision Number (Float)	Sensitivity Number (Float)	Specificity Number (Float)	F-measure Number (Float)	Accuracy Number (Float)	Cohen's ka.. Number (Flo
1	HIGH...	12	0	8	0	1	1	1	1	1	?	?
2	BALA...	7	1	12	0	1	0.875	1	0.923	0.933	?	?
3	LIGHT	0	0	19	1	0	?	0	1	?	?	?
4	Overall	?	?	?	?	?	?	?	?	?	0.95	0.9

ACCURACY: 95%
PRECISION: 1 (HIGH-CALORIE), 1 (BALANCED), MISSING VALUE (LIGHT)
RECALL: 1 (HIGH-CALORIE), 1 (BALANCED), 0 (LIGHT)
F-1 SCORE/F-MEASURE: 1 (HIGH-CALORIE), 0.923 (BALANCED), MISSING VALUE (LIGHT)

WORKFLOW RESULTS

DECISION TREE SCORER CONFUSION MATRIX

▶ 1: Confusion matrix ▶ 2: Accuracy statistics Flow Variables

Rows: 3 | Columns: 3 Sorting Filter Table Statistics Search

#	RowID	HIGH-CALORIE Number (Integer)	BALANCED Number (Integer)	LIGHT Number (Integer)
1	HIGH...	12	0	0
2	BALA...	0	7	0
3	LIGHT	0	1	0

ACCURACY: 95%
PRECISION: 1 (HIGH-CALORIE), 1 (BALANCED), MISSING VALUE (LIGHT)
RECALL: 1 (HIGH-CALORIE), 1 (BALANCED), 0 (LIGHT)
F-1 SCORE/F-MEASURE: 1 (HIGH-CALORIE), 0.93, MISSING VALUE (LIGHT)

WORKFLOW RESULTS

RANDOM FOREST SCORER

Rows: 4 | Columns: 11 Sorting Filter Search

#	RowID	TruePositiv... Number (Inte...	FalsePositi... Number (Inte...	TrueNegati... Number (Inte...	FalseNegat... Number (Inte...	Recall Number (Float)	Precision Number (Float)	Sensitivity Number (Float)	Specificity Number (Float)	F-measure Number (Float)	Accuracy Number (Float)	Cohen's ka.. Number (Flo
1	HIGH...	11	1	7	1	0.917	0.917	0.917	0.875	0.917	?	?
2	BALA...	6	2	11	1	0.857	0.75	0.857	0.846	0.8	?	?
3	LIGHT	0	0	19	1	0	?	0	1	?	?	?
4	Overall	?	?	?	?	?	?	?	?	?	0.85	0.7

ACCURACY: 85%
PRECISION: 0.917 (HIGH-CALORIE), 0.75 (BALANCED), MISSING VALUE (LIGHT)
RECALL: 0.917 (HIGH-CALORIE), 0.857 (BALANCED), 0 (LIGHT)
F-1 SCORE/F-MEASURE: 0.917 (HIGH-CALORIE), 0.8 (BALANCED), MISSING VALUE (LIGHT)

WORKFLOW RESULTS

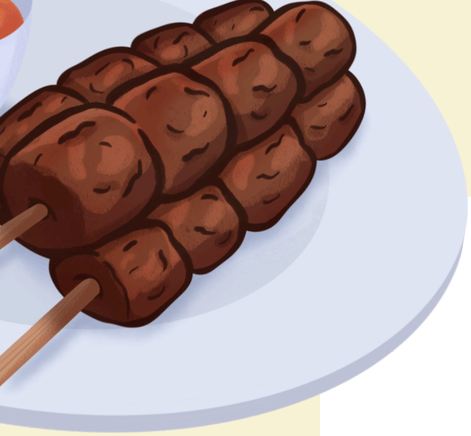
RANDOM FOREST SCORER CONFUSION MATRIX

▶ 1: Confusion matrix ▶ 2: Accuracy statistics Flow Variables

Rows: 3 | Columns: 3 Sorting Filter Table Statistics Search

#	RowID	HIGH-CALORIE Number (Integer)	BALANCED Number (Integer)	LIGHT Number (Integer)
1	HIGH...	11	1	0
2	BALA...	1	6	0
3	LIGHT	0	1	0

ACCURACY: 85%
PRECISION: 0.917 (HIGH-CALORIE), 0.75 (BALANCED), MISSING VALUE (LIGHT)
RECALL: 0.917 (HIGH-CALORIE), 0.857 (BALANCED), 0 (LIGHT)
F-1 SCORE/F-MEASURE: 0.917 (HIGH-CALORIE), 0.8 (BALANCED), MISSING VALUE (LIGHT)

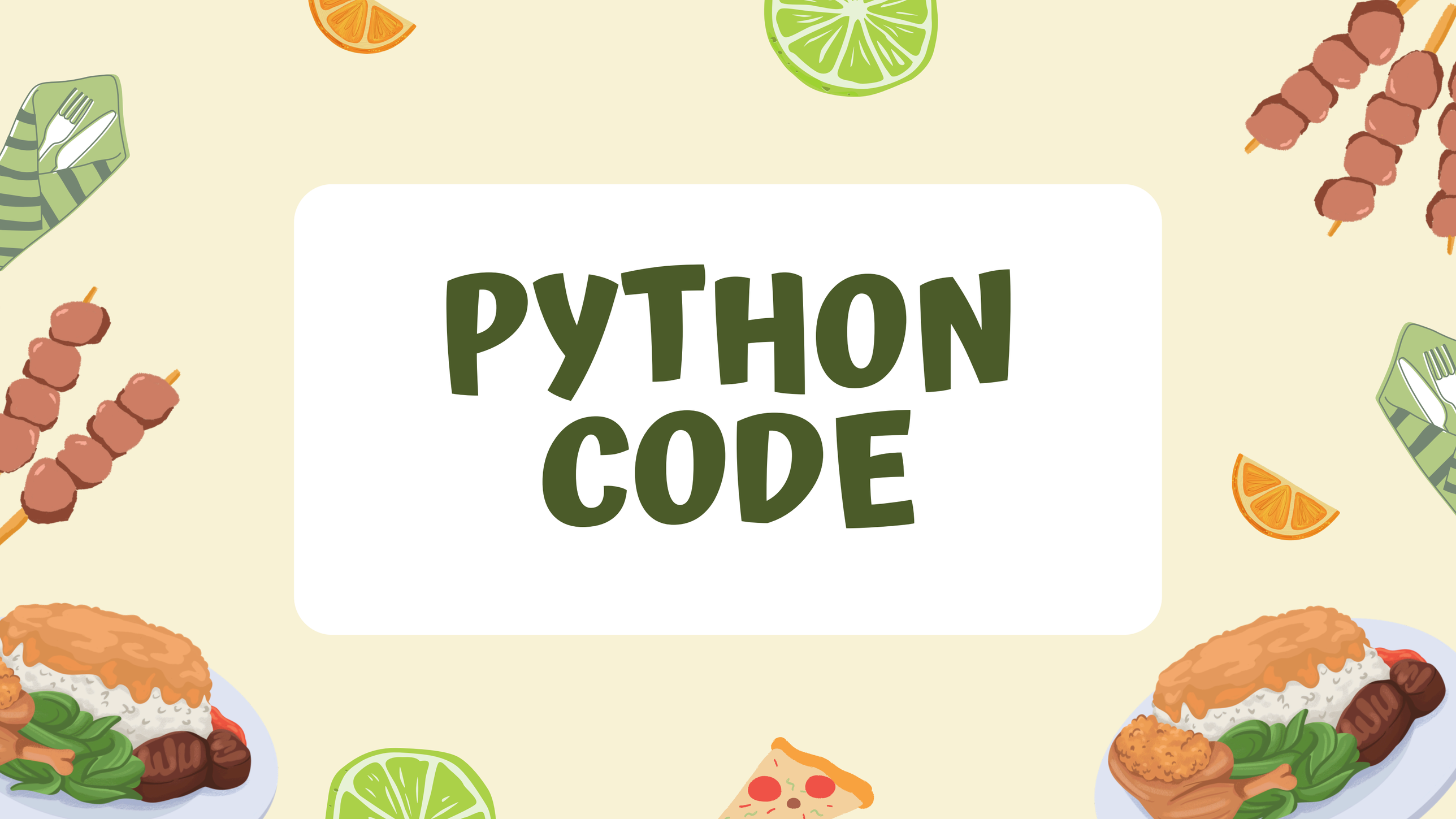


SYSTEM QUESTION

BASED ON THE RECORDED INPUTS, WHICH CATEGORY SHOULD THE SYSTEM ASSIGN FOR CAFETERIA MEAL CATEGORY CLASSIFICATION?

USING KNIME RESULTS AS REFERENCE, IN PRACTICE IT IS RECOMMENDED THAT

- **HIGH-CALORIE:** CONSISTENTLY ASSIGNED FOR DENSE MEALS WITH HIGH CONFIDENCE (92%–100% PRECISION/RECALL ACROSS BOTH MODELS).
- **BALANCED:** RELIABLY ASSIGNED FOR MODERATE MEALS, BUT ACTS AS A FALLBACK FOR AMBIGUOUS CASES.
- **LIGHT:** NEVER ASSIGNED BY EITHER MODEL (0 PREDICTIONS RECORDED; 0% RECALL). SYSTEM BEHAVIOR: ANY MEAL INTENDED TO BE CLASSIFIED AS LIGHT DEFAULTS TO BALANCED DUE TO SEVERE CLASS IMBALANCE.


PYTHON
CODE

CODE

MARQUEZ_RESPICIO_MIDTERMS.py | streamlit.py

MARQUEZ_RESPICIO_MIDTERMS.py > ...

```python
# PART 4: RECREATE THE PROCESS IN PYTHON

import pandas as pd
import joblib
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (accuracy_score, precision_score,
                             recall_score, f1_score, confusion_matrix)

print("\nProgrammed by: Arianne Beverly Marquez & Lester Anthony Respicio\n")

print("\nInitializing and loading csv...\n")
```

CODE

```
16
17  # 1. Load the Dataset
18  file_path = "MARQUEZ_RESPICIO_CS0065 MIDTERM - Sheet1.csv"
19  df = pd.read_csv(file_path)
20
21  # 2. Exclude identifiers & Prepare X (Inputs) and y (Target)
22  feature_cols = ['CALORIES (kcal)', 'SUGAR (g)', 'FAT (g)', 'PROTEIN (g)', 'SODIUM (mg)']
23  X = df[feature_cols]
24  y = df['CATEGORY']
25
26  # 3. Stratified Partitioning (80/20 matching Table Partitioner)
27  X_train, X_test, y_train, y_test = train_test_split(
28      X, y,
29      test_size=0.20,
30      stratify=y,
31      random_state=42
32  )
33
```

CODE

```
33
34  # 4. Define and Train Pipelines (Handling Median Imputation + Model Training)
35  dt_pipeline = Pipeline([
36      ('imputer', SimpleImputer(strategy='median')),
37      ('classifier', DecisionTreeClassifier(criterion='gini', min_samples_split=2, random_state=42))
38  ])
39  dt_pipeline.fit(X_train, y_train)
40  dt_predictions = dt_pipeline.predict(X_test)
41
42  rf_pipeline = Pipeline([
43      ('imputer', SimpleImputer(strategy='median')),
44      ('classifier', RandomForestClassifier(n_estimators=100, criterion='gini', random_state=42))
45  ])
46  rf_pipeline.fit(X_train, y_train)
47  rf_predictions = rf_pipeline.predict(X_test)
48
```

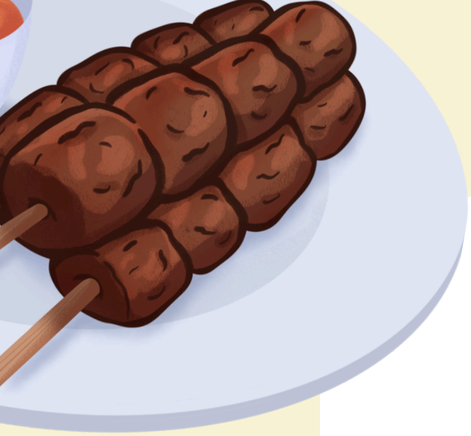
CODE

```
48
49
50  # --- EVALUATION OUTPUT ---
51
52  def evaluate_model(name, y_true, y_pred):
53      acc = accuracy_score(y_true, y_pred)
54      prec = precision_score(y_true, y_pred, average='weighted', zero_division=0)
55      rec = recall_score(y_true, y_pred, average='weighted', zero_division=0)
56      f1 = f1_score(y_true, y_pred, average='weighted', zero_division=0)
57
58      print(f"{name.upper()} EVALUATION")
59      print(f"{'='*35}")
60      print(f"{'Metric':<15} | {'Score':<15}")
61      print(f"{'-'*15}-+-{'-'*15}")
62      print(f"{'Accuracy':<15} | {acc:.4f}")
63      print(f"{'Precision':<15} | {prec:.4f}")
64      print(f"{'Recall':<15} | {rec:.4f}")
65      print(f"{'F1-Score':<15} | {f1:.4f}")
66      print(f"{'='*35}\n")
67
68      print("Confusion Matrix:")
69      print(confusion_matrix(y_true, y_pred))
70      print("\n")
71
72  evaluate_model("Decision Tree", y_test, dt_predictions)
73  evaluate_model("Random Forest", y_test, rf_predictions)
74
75  # 5. Save the Selected Pipeline for Streamlit (Part 5)
76  joblib.dump(dt_pipeline, 'meal_category_model.pkl')
77  print("Pipeline saved successfully as 'meal_category_model.pkl'")
```

OUTPUT

```
Programmed by: Arianne Beverly Marquez & Lester Anthony Respicio


Initializing and loading csv...

DECISION TREE EVALUATION
==================================
Metric          | Score
----------------+----------------
Accuracy        | 0.9500
Precision       | 0.9062
Recall          | 0.9500
F1-Score        | 0.9267
==================================

Confusion Matrix:
[[ 7  0  0]
 [ 0 12  0]
 [ 1  0  0]]
```

```
RANDOM FOREST EVALUATION
==================================
Metric          | Score
----------------+----------------
Accuracy        | 0.9000
Precision       | 0.8538
Recall          | 0.9000
F1-Score        | 0.8760
==================================

Confusion Matrix:
[[ 6  1  0]
 [ 0 12  0]
 [ 1  0  0]]


Pipeline saved successfully as 'meal_category_model.pkl'
PS C:\Knime\CS0065\MARQUEZ_RESPICIO_MIDTERM>
```

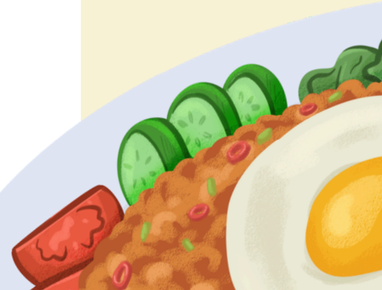


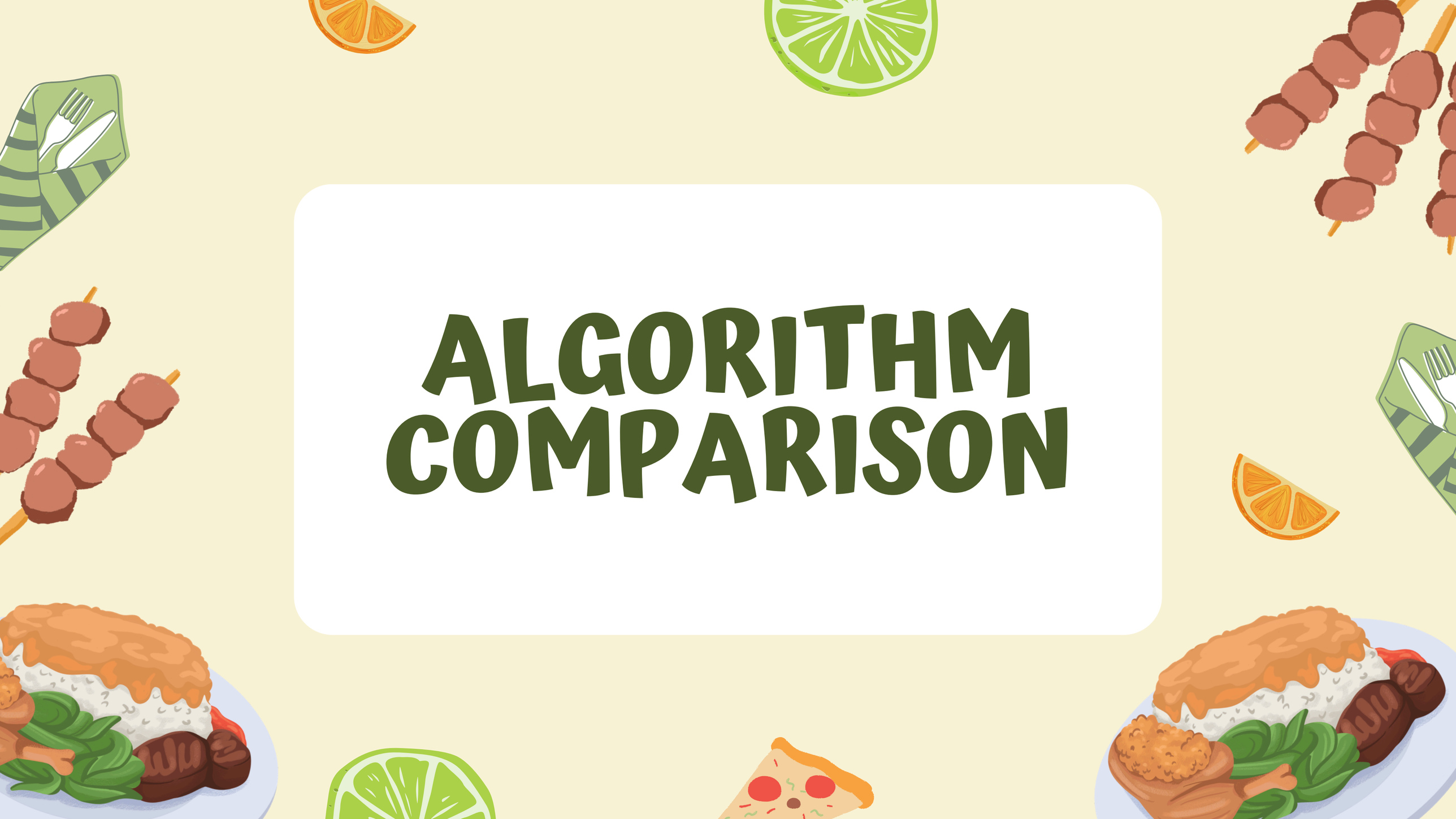
SYSTEM QUESTION

BASED ON THE RECORDED INPUTS, WHICH CATEGORY SHOULD THE SYSTEM ASSIGN FOR CAFETERIA MEAL CATEGORY CLASSIFICATION?

USING PYTHON PROGRAM OUTPUT AS REFERENCE, IN PRACTICE IT IS RECOMMENDED THAT

- **HIGH-CALORIE**: CONSISTENTLY AND PERFECTLY ASSIGNED FOR DENSE MEALS (100% RECALL ACROSS BOTH MODELS).
- **BALANCED**: RELIABLY ASSIGNED FOR MODERATE MEALS, BUT CONTINUES TO ACT AS A FALLBACK FOR THE UNDERREPRESENTED CLASS.
- **LIGHT**: RARELY TO NEVER ASSIGNED ACCURATELY. THE SINGLE "LIGHT" MEAL IN THE TEST DATA WAS MISCLASSIFIED AS "BALANCED" BY BOTH MODELS DUE TO SEVERE CLASS IMBALANCE.





ALGORITHM COMPARISON

ALGORITHM COMPARISON

METRICS	DECISION TREE (PRIMARY ALGORITHM)	RANDOM FOREST (COMPARISON ALGORITHM)
ACCURACY	95%	90%
PRECISION	90.62	85.38
RECALL	95%	90%
F-1 MEASURE/F-1 SCORE	92.67	87.60

STREAMLIT INTERFACE

CODE

MARQUEZ_RESPICIO_MIDTERMS.py | streamlit.py

streamlit.py > ...

```python
import streamlit as st
import pandas as pd
import joblib
import base64

# --- BACKGROUND SETUP ---
def set_background(image_file):
    try:
        with open(image_file, "rb") as file:
            encoded_string = base64.b64encode(file.read()).decode()

        # CSS to inject the background and add a frosted cream overlay for readability
        css = f"""
        <style>
        .stApp {{
            background-image: url({encoded_string});
            background-size: 400px; /* Sets a fixed scale instead of stretching */
            background-repeat: repeat; /* Tiles the image across the screen */
            background-attachment: fixed;
        }}
        .block-container {{
            background-color: rgba(241, 235, 225, 0.95);
            border-radius: 15px;
            padding: 2rem;
            margin-top: 2rem;
            box-shadow: 0 4px 6px rgba(0, 0, 0, 0.1);
        }}
        </style>
        """
        st.markdown(css, unsafe_allow_html=True)
    except FileNotFoundError:
        pass

```

CODE

```
33
34  # Call the background function
35  set_background('bg2.jpg')
36
37  # --- INITIALIZE STATE ---
38  if 'classified' not in st.session_state:
39      st.session_state.classified = False
40  if 'prediction' not in st.session_state:
41      st.session_state.prediction = None
42  if 'is_zero_nutrient' not in st.session_state:
43      st.session_state.is_zero_nutrient = False
44
45  # --- LOAD MODEL ---
46  try:
47      model = joblib.load('meal_category_model.pkl')
48  except FileNotFoundError:
49      st.error("Model file not found. Please ensure 'meal_category_model.pkl' is in the same folder.")
50      st.stop()
51
52  st.title("Cafeteria Meal Category Classifier")
53  st.write("Enter the nutritional information of a meal to classify it as Light, Balanced, or High-Calorie.")
```

CODE

```
54
55  # --- INPUT LAYOUT ---
56  col1, col2 = st.columns(2)
57
58  with col1:
59      calories = st.number_input("Calories (kcal)", min_value=0.0, step=10.0, key="cal_input")
60      sugar = st.number_input("Sugar (g)", min_value=0.0, step=1.0, key="sugar_input")
61      fat = st.number_input("Fat (g)", min_value=0.0, step=1.0, key="fat_input")
62
63  with col2:
64      protein = st.number_input("Protein (g)", min_value=0.0, step=1.0, key="protein_input")
65      sodium = st.number_input("Sodium (mg)", min_value=0.0, step=10.0, key="sodium_input")
66
67  st.write("")
68
69
```

CODE

```
69
70  # --- CLASSIFY BUTTON & VALIDATION ---
71  btn_col1, btn_col2, btn_col3 = st.columns([1, 1, 1])
72  with btn_col2:
73      if st.button("Classify Meal", use_container_width=True):
74          # 1. Check for invalid negative values
75          if any(val < 0 for val in [calories, sugar, fat, protein, sodium]):
76              st.error("Invalid input: Nutritional values cannot be negative.")
77              st.session_state.classified = False
78          else:
79              # Set flag for zero-nutrient entry
80              st.session_state.is_zero_nutrient = (
81                  calories == 0 and (sugar + fat + protein + sodium) == 0
82              )
83
84              input_data = pd.DataFrame({
85                  'CALORIES (kcal)': [calories],
86                  'SUGAR (g)': [sugar],
87                  'FAT (g)': [fat],
88                  'PROTEIN (g)': [protein],
89                  'SODIUM (mg)': [sodium]
90              })
91
92              st.session_state.prediction = model.predict(input_data)[0]
93              st.session_state.classified = True
94
```

CODE

```
94
95  # --- FULL-WIDTH NOTICE ---
96  # Outside the columns so it stretches the full width of the container
97  if st.session_state.classified and st.session_state.is_zero_nutrient:
98      st.write("")
99      st.info("Notice: All values are 0 (e.g., plain water / zero-calorie beverage).")
100
101 # --- RESULTS DISPLAY ---
102 if st.session_state.classified and st.session_state.prediction is not None:
103     st.markdown("---")
104     pred_str = str(st.session_state.prediction).strip()
105     st.subheader(f"Result: {pred_str}")
106
```

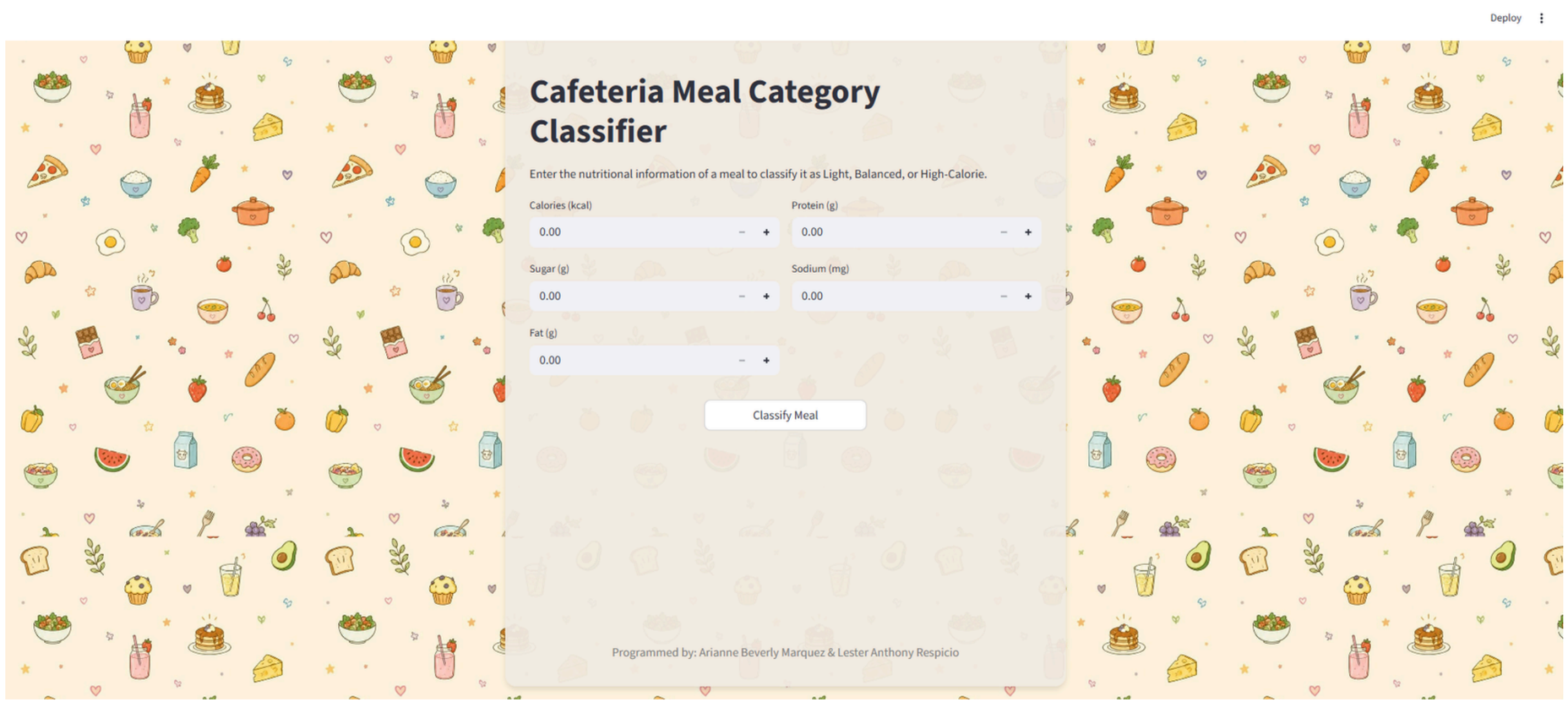
CODE

```
106
107     # Normalized to upper case to prevent string mismatch
108     pred_upper = pred_str.upper()
109     if "HIGH" in pred_upper:
110         st.error("Explanation: This meal contains a high density of calories and macronutrients, suitable for intensive energy rec
111     elif "BALANCED" in pred_upper:
112         st.success("Explanation: This meal provides a well-proportioned mix of nutrients, ideal for maintaining steady energy leve
113     elif "LIGHT" in pred_upper:
114         st.warning("Explanation: This meal is low in calories and heavy macronutrients, making it a great option for a quick snack
115     else:
116         st.info("Explanation: Meal category successfully classified.")
117
118     st.write("")
119
```

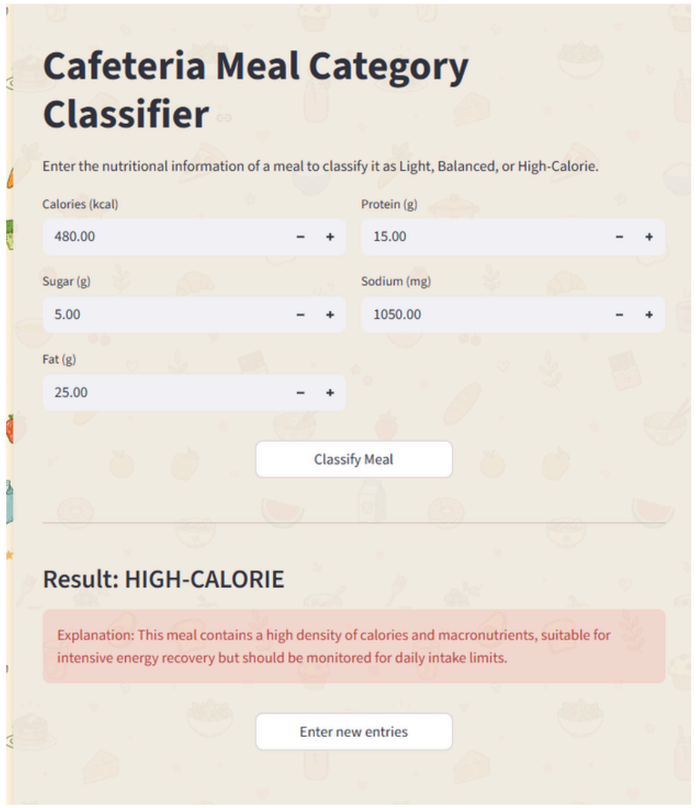
CODE

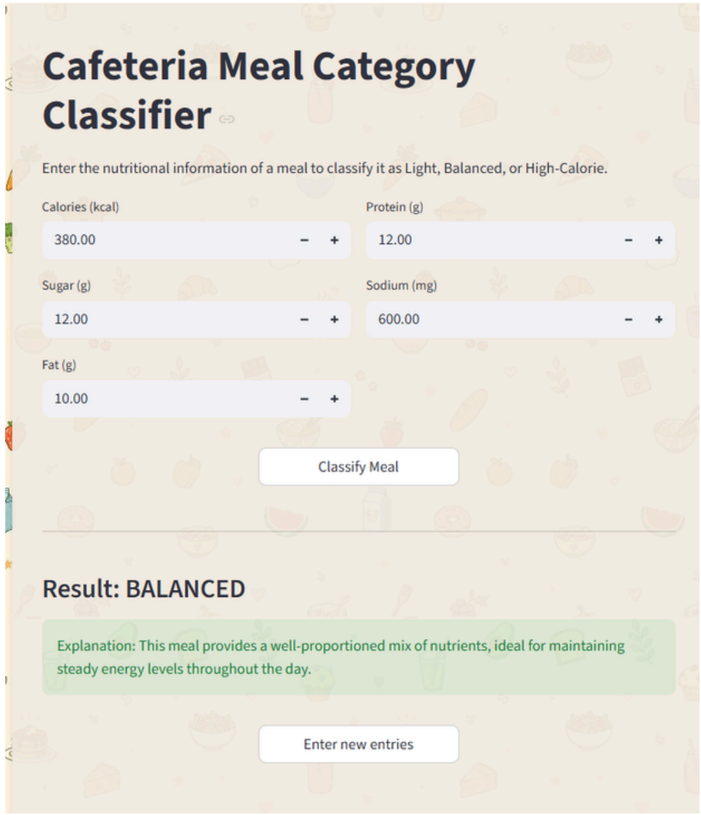
```
119
120     def reset_inputs():
121         st.session_state.classified = False
122         st.session_state.prediction = None
123         st.session_state.cal_input = 0.0
124         st.session_state.sugar_input = 0.0
125         st.session_state.fat_input = 0.0
126         st.session_state.protein_input = 0.0
127         st.session_state.sodium_input = 0.0
128
129     reset_col1, reset_col2, reset_col3 = st.columns([1, 1, 1])
130     with reset_col2:
131         st.button("Enter new entries", on_click=reset_inputs, use_container_width=True)
132 else:
133     st.write("<br>" * 7, unsafe_allow_html=True)
134
135 # --- FOOTER ---
136 st.markdown("<br><br>", unsafe_allow_html=True)
137 st.markdown("<p style='text-align: center; color: gray;'>Programmed by: Arianne Beverly Marquez & Lester Anthony Respicio</p>", unsafe_allow_ht
```

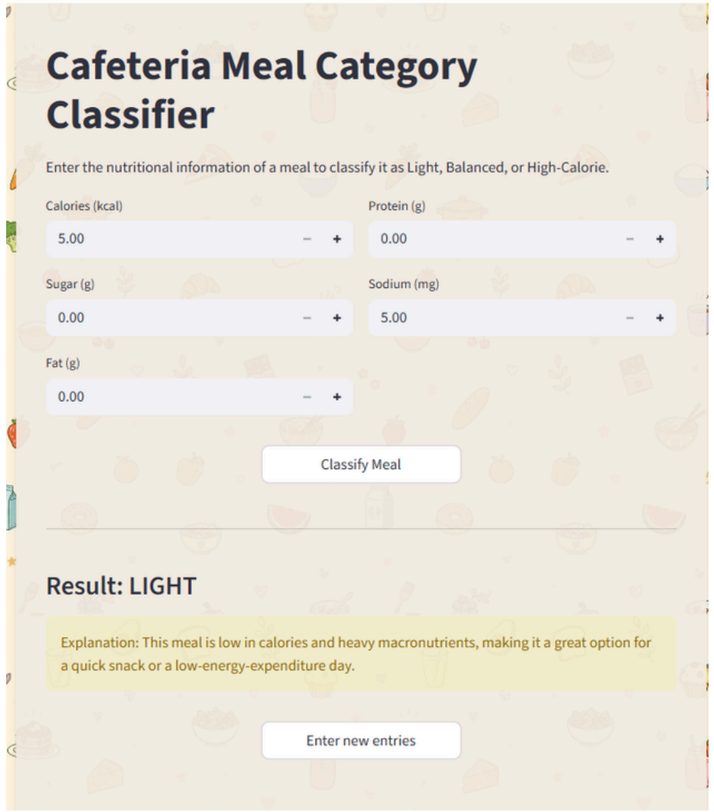
OUTPUT


Deploy
Cafeteria Meal Category Classifier
Enter the nutritional information of a meal to classify it as Light, Balanced, or High-Calorie.
Calories (kcal)
0.00
Protein (g)
0.00
Sugar (g)
0.00
Sodium (mg)
0.00
Fat (g)
0.00
Classify Meal
Programmed by: Arianne Beverly Marquez & Lester Anthony Respicio

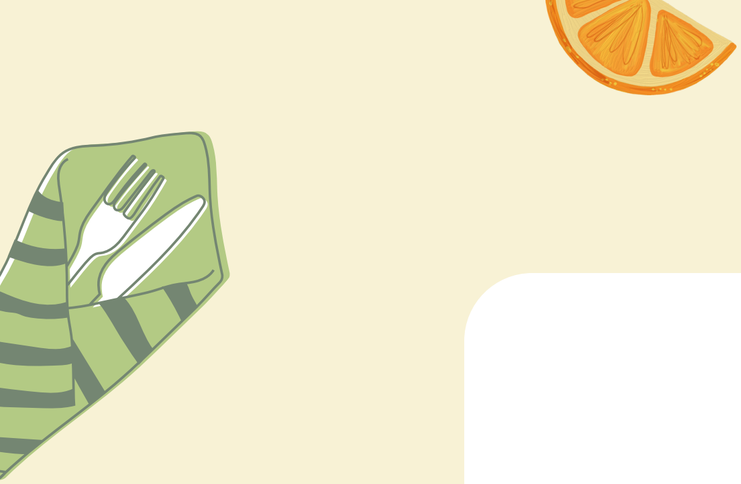
OUTPUT

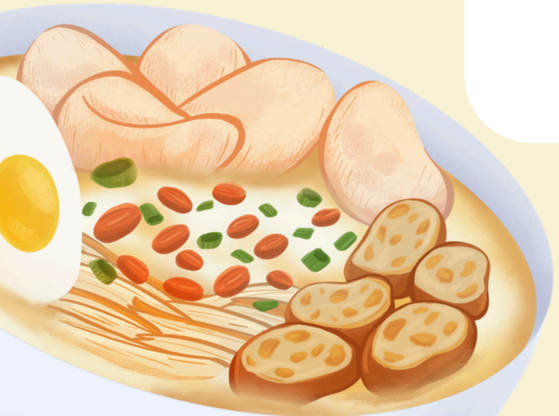

Cafeteria Meal Category Classifier
Enter the nutritional information of a meal to classify it as Light, Balanced, or High-Calorie.
Calories (kcal)
480.00
Protein (g)
15.00
Sugar (g)
5.00
Sodium (mg)
1050.00
Fat (g)
25.00
Classify Meal
Result: HIGH-CALORIE
Explanation: This meal contains a high density of calories and macronutrients, suitable for intensive energy recovery but should be monitored for daily intake limits.
Enter new entries

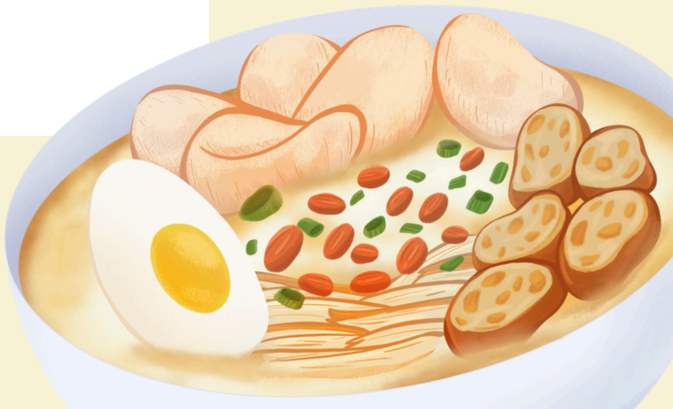

Cafeteria Meal Category Classifier
Enter the nutritional information of a meal to classify it as Light, Balanced, or High-Calorie.
Calories (kcal)
380.00
Protein (g)
12.00
Sugar (g)
12.00
Sodium (mg)
600.00
Fat (g)
10.00
Classify Meal
Result: BALANCED
Explanation: This meal provides a well-proportioned mix of nutrients, ideal for maintaining steady energy levels throughout the day.
Enter new entries


Cafeteria Meal Category Classifier
Enter the nutritional information of a meal to classify it as Light, Balanced, or High-Calorie.
Calories (kcal)
5.00
Protein (g)
0.00
Sugar (g)
0.00
Sodium (mg)
5.00
Fat (g)
0.00
Classify Meal
Result: LIGHT
Explanation: This meal is low in calories and heavy macronutrients, making it a great option for a quick snack or a low-energy-expenditure day.
Enter new entries

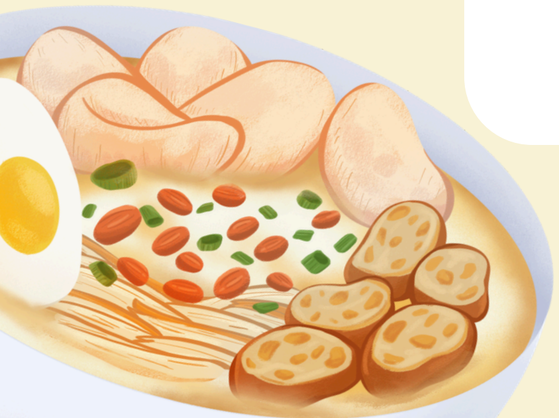
TEST, EXPLAIN & REFLECT

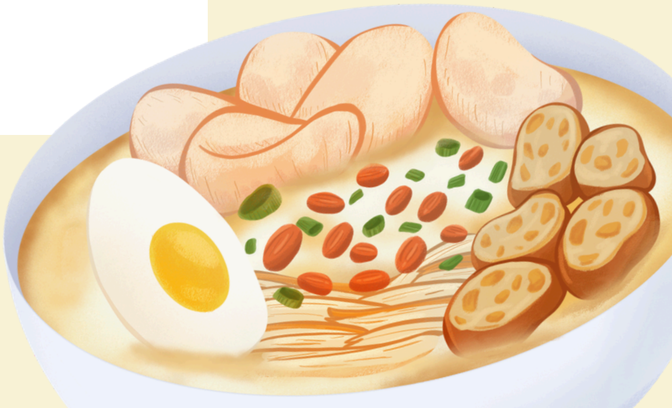






TESTING





EXPLAIN

- Which input features should have the strongest influence on the assigned category?
    - **Calories and fat** drive energy density classification, while protein distinguishes balanced meals from light or empty high-calorie items.
- What should the system do when the available evidence is incomplete or contradictory?
    - The pipeline applies **median imputation** for missing data, runs validation checks against physical inconsistencies, and flags uncertain predictions for review.
- What could happen if the system produces an incorrect result for cafeteria meal category classification?
    - Inaccurate labels can **compromise health management** for dietary conditions, which may lead to under-fueling, and erode consumer trust in the system.

REFLECTION

- Implementing the classifier in both KNIME and Python demonstrated high effectiveness for High-Calorie and Balanced meals, but exposed a key limitation with severe class imbalance causing the "Light" category to be ignored.
- This highlighted that raw model accuracy is insufficient without assessing minority recall, especially in dietary systems where mislabeled nutritional information impacts consumer health.
- Moving forward, refining the dataset and threshold rules will ensure more equitable and reliable real-world recommendations.

THANK
YOU!